

自进化 (Self-evolving / RSI)，一篇就够了

关于作者周星星算法工程师，7年算法应用实践，3年大模型应用实践回答34文章82关注者
5,327关注他发私信：61-77 minutes

这几个月，AI 圈子的热词肯定少不了 self-evolving、self-improving、recursive self-improvement (RSI) 这几个概念。

模型能自己优化自己，这么酷的一件事，我相信也是我们这些从业者相信会发生的方向——经历过去一年模型智能+快速跃进的人，应该都有同感。只不过眼下这些工作还处于早期阶段。

不同人对"自进化+"的理解不太一样。这篇文章想把这事讲清楚：先看看前沿机构实际在做什么，然后我们再给自进化下一个清晰的定义，最后梳理几个具体的研究方向。

自进化，一篇就够了！

机构的”自进化“

AI 两巨头，OpenAI+ 随 GPT-5.6 一起放出了个新指标 “RSI Index+”，专门衡量模型自己搞研究的能力，最强档 Sol 比上一代 GPT-5.5 高了 16.2 分，官方给的

案例是它自己选训练配置、自己跑完 post-training，直接训出了更小的 Luna+。**Anthropic** 的《When AI builds itself》按时间线拆了五个阶段，具体提到的机制包括代码生成+、代码审查、实验设计等，给的数字是 Claude+ 现在写了公司 80%+ 的合入代码，而且它能独立搞定的任务时长大概每 4 个月就翻一倍。

国内这边。腾讯混元 7 月 21 日发的 Hyra-1.0 已经能自己跑”探索 → 提方案 → 读反馈 → 修订”这套循环；MiniMax 4 月开源的 M2.7，官方称是自己”第一个深度参与自我进化的模型”，在内部 harness 里自主跑了 100 多轮”分析失败 → 改代码 → 跑评测”，把性能提了 30%。

创业公司这边就是押注了。田渊栋（原 Meta FAIR+ 研究总监）今年 5 月加入了 Richard Socher（前 Salesforce 首席科学家）牵头成立的 Recursive Superintelligence，几个月就融了 6.5 亿美元、估值冲到 46.5 亿美元，押的是用 latent token 取代语言 token 让模型在 latent space 这个更无损的空间做自我改进。Sakana AI（由 David Ha 和 Transformer+ 论文作者之一 Llion Jones 在东京创立）6 月开了专门的 RSI Lab+，走的是演化算法+这条路。Apodex 是盛大网络+创始人陈天桥创立的公司，靠上百个子 agent 分工协同、外加独立验证团队互相纠错；官方管这叫”discoverative intelligence”。Weco AI 是一支专注 autoresearch 方向的小团队，官网口号是”We build recursively self-improving AI”。他们最近做的 AIDE² 实验，用一个双层优化让 agent 自己改进自己做研究的 agent，8 天、100 步迭代出的版本在三个外部基准上都显著超过基线。

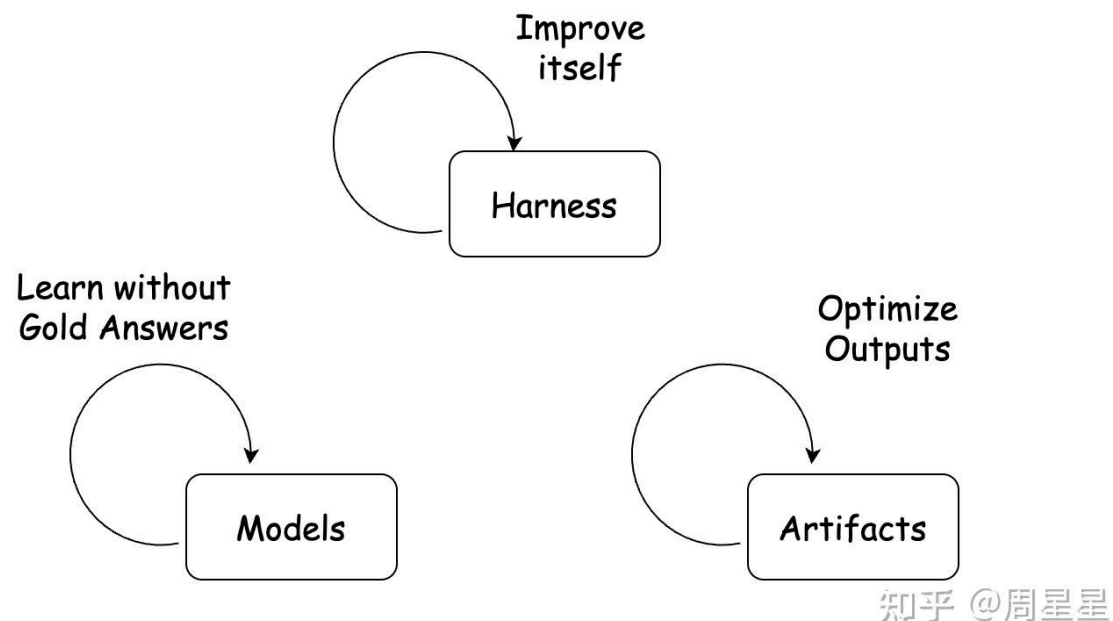
问题来了～

什么是 Self-evolving / self-improving / RSI

个人比较喜欢 A Taxonomy of Self-Evolving Agents 中的定义。

自进化可以分成以下三个方向，三个方形都是自进化。

自我进化的目标优化对象是什么，就决定了它属于哪一个方向——改产出物（代码/论文/算法...）是 Artifacts，改脚手架（prompt/memory/tool/skill/hook...）是 Harness，改模型参数本身是 Model。三者都算 RSI：



RSI 三层分类：Harness / Models / Artifacts

1. **Artifacts 层面的自进化**：用强大的 LLM 反复“发现问题 → 生成产出 → 评估结果”，来优化一个具体的复杂问题产物（代码、论文、算法）。比如任何一款 AI coding 工具⁺，给定一个目标，它就会写代码、自己编译、自己测试，有 bug 自己修，最终输出能达到要求的代码。产出物一版比一版强，这就是 Artifacts 层面的自我进化，改的是产出物本身，不涉及模型权重，也不涉及 agent 自身的脚手架逻辑。
2. **Harness 层面的自进化**：不动模型权重，改的是部署后 Agent 本身的“脚手架”：prompt、memory、tool、skill、多 Agent 路由等等等等都是这一类。跟 Artifacts 层不一样的地方在于，Artifacts 优化的是某一次任务的产出物；Harness 优化的是 agent 下次执行任何任务时都会用到的那套“脚手架”，改一次，后面所有任务都受益。最直观的例子是 Agent 跑任务踩了坑，把错误总结成一条新 skill 或一段新 memory 存下来，下次碰到同类任务直接调用，不用再犯一次。agent 自己把自己的脚手架越改越顺手，这就是 Harness 层面的自我进化。
3. **模型层面的自进化**：广义的角度，不需要人工标注答案、模型自己给自己当老师的训练都能算：自己筛出高置信度/前后一致的答案反过来当训练数据（self-training⁺、TTRL）、把内部信号转成可验证奖励再用 RL 训练（DeepSeek-R1）、两个版本的自己互相对练（自对弈，例如 SPIN、Absolute Zero）、推理时当场再学一遍（测试时训练）。不靠人喂标注答案，模型自己给自己当老师，这就是 Model 层面的自我进化。狭义的层面，是模型能自己产出下一代的训练优化方向，一代代往上迭代——模型自己训练完 → 自己测试 → 自己找问题（架构瓶颈在哪、工程实现哪里有

坑、训练数据有什么问题) → 自己提出优化方向/实验 (例如自己改训练代码, 自己补充训练数据) → 再训练, 整个研发循环都由 AI 自己跑。

这三个层面发生的事情都是自我进化, **区别是优化目标不一样, 共同点是模型都在一个 loop 中不断自我改进, 不断迭代取得更好的结果。**

前两者都是 train-free, 三层的边界正在模糊、也在相互反哺。Harness 里积累的经验能变成训练数据、变成更好的训练基建脚手架, 这些数据和训练脚手架反过来让训练后的基模学会更好地做自进化的任务; 模型和 harness 都变强后, 又能产出更好的 artifacts, 这些 artifacts 又能变成 harness 的新工具。

这三个方向最终变成一个闭环, 谁不喜欢闭环呢?

实现的路径

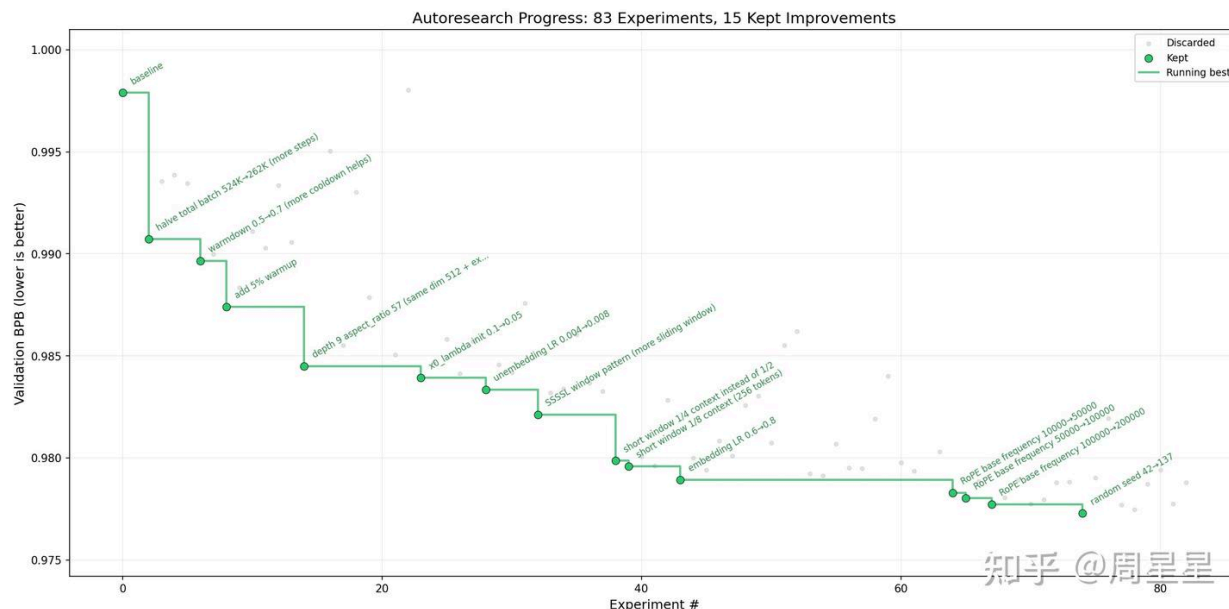
Artifacts 层面

这一层面, 是让模型自己迭代输出一个更好的 artifacts, 这里举两个例子, 一个是 autoresearch 产出更好的模型训练代码/模型超参⁺, 另外一个产出更好的 infra 算法。

Karpathy⁺ 的 Autoresearch: 整夜自动改 train.py 迭代超参

给 agent 配一个小型但真实的 LLM 训练设置, 让它整夜自己做实验: agent 只能改一个文件 train.py (架构、超参、优化器、batch size 随便调), 每次固

定跑 5 分钟训练，用 val_bpb (验证集+ bits-per-byte, 越低越好) 这个客观指标打分，改好了就留、没改好就扔，自动循环。按这个节奏一小时能跑 12 个实验，一夜下来能跑上百个。这里的 Artifacts，就是那个被不断改进的 train.py/模型配置本身。

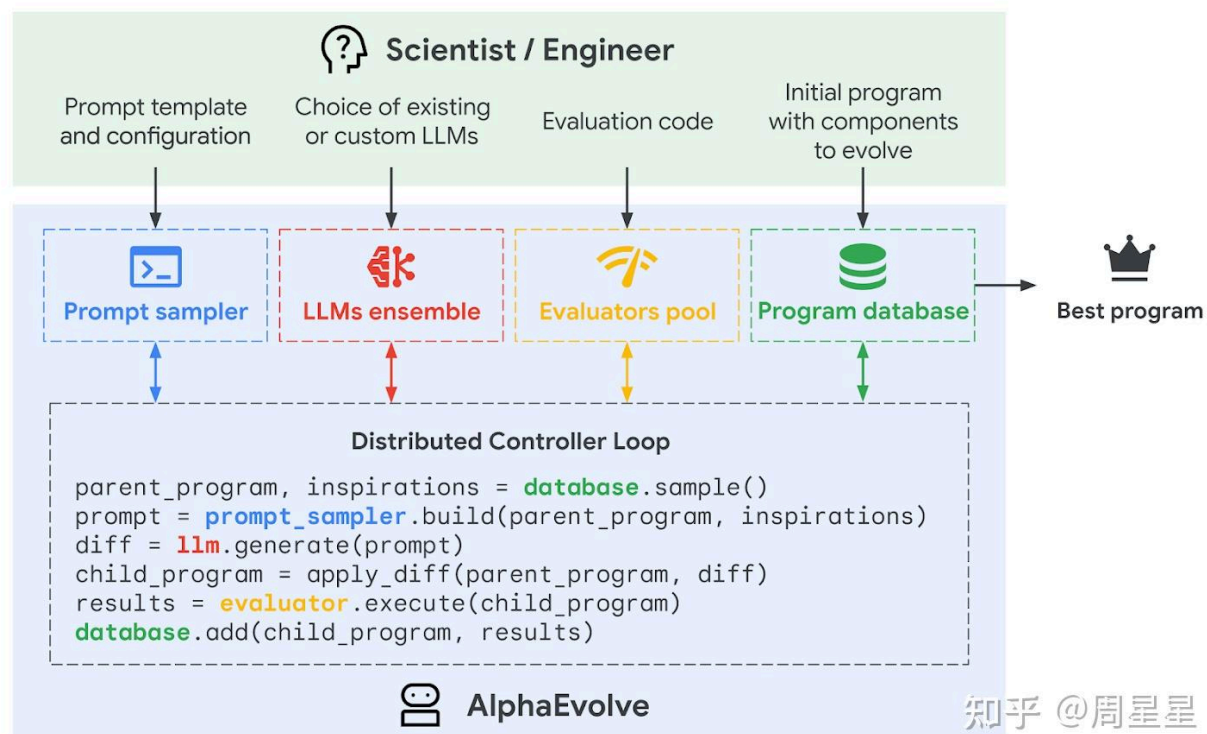


Karpathy 自己在推特上贴过跑了约两天、depth=12 模型的结果：agent 自主尝试了约 700 次改动，其中约 20 次真正带来提升被保留下来，把训练到 GPT-2 水平所需的时间从 2.02 小时压到了 1.80 小时，快了大概 11%。

Google DeepMind+ 的 AlphaEvolve：进化算法筛选代码，成果反哺自身训练

用 Gemini 生成候选代码，配合自动评估器打分，再用进化算法保留最优的那批，循环产出更好的算法——这就是 AlphaEvolve 的思路。

它已经在生产环境里跑了一年多：帮 Gemini 自己的矩阵乘法+核心提速 23%、FlashAttention+ 提速 32.5%，甚至提出过硬件层面的 Verilog 修改。这些改进又被用回 Gemini 的训练里，形成一个“AI 优化驱动自己的模型”的闭环。目前每轮迭代大概能省下 1% 左右的计算，业内讨论认为这个数字一旦冲到 5%-10%，“自我改进”的时间表会从几十年压缩到几年。AlphaEvolve 也因此常被当成 RSI 已经在生产环境里悄悄发生的例证。



AlphaEvolve 的进化流水线：prompt sampler 取样历史程序 → LLM 生成候选代码 → 自动评估器打分 → 进化算法保留最优个体，循环迭代

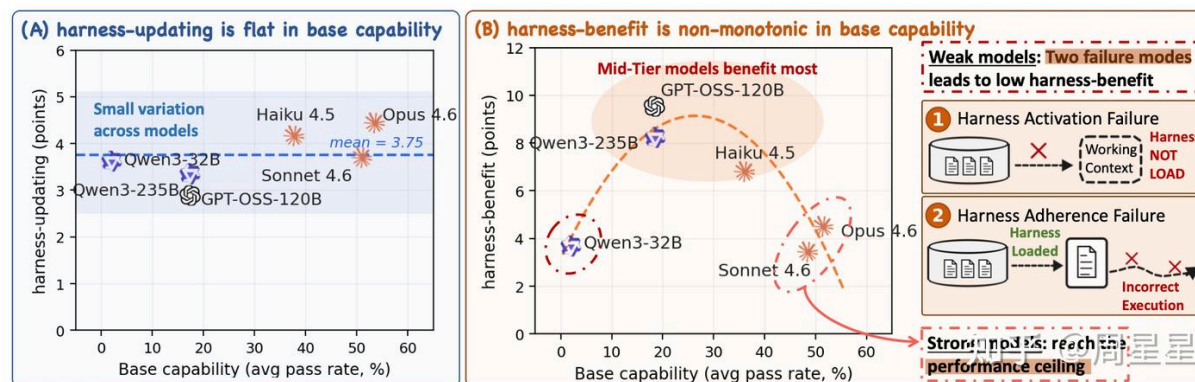
Harness 层面：万花齐放

翁荔+最新博客《Harness Engineering for Self-Improvement》里判断，RSI 近期不太可能从模型直接改写自己的权重开始，更现实的路径是先**Harness 层爆发**。理由两条：

1. **改 Harness 本身就能省钱**：几轮 Harness 迭代就能把推理成本最多打下来 60%，靠的是把上下文管理得更好、agent 之间配合更顺，不是靠模型多想（拉长推理链条+）。
2. **提建议不挑模型，真正获益的是大规模部署的中间档模型**：翁荔博客里提到，Lin et al. (2026) 把”harness-updating”（提出有用改动的能力）和”harness-benefit”（用好新 harness 的能力）这两条轴拆开测，结果是从 Qwen3-32B 到 Opus 4.6 这一整排模型，harness-updating 能力几乎是一条平线，波动很小；但 harness-benefit（能不能真把新 harness 用好）是非单调的，像 GPT-OSS-120B、Qwen3-235B 这类中间档位的模型获益最大，弱模型容易卡在”harness 都没加载进去”或”加载了但执行错”这两种失败模式，强模型则很快撞到自己的能力天花板。

这两条结论合起来解释了现在做 harness 自进化为什么划算：一是提改动建议这件事不挑模型，不用烧最贵的旗舰模型去设计 harness，用便宜的中等模型就

能干；二是真正吃到红利的恰恰是中间档位模型，而生产环境里大部分公司部署的就是这类模型，不是每个任务都上最贵最强的。



(A) harness-updating 能力在 Qwen3-32B 到 Opus 4.6 之间基本持平；(B) harness-benefit 呈非单调曲线，GPT-OSS-120B、Qwen3-235B 等中间档位模型获益最大，弱模型受限于两类失败模式、强模型撞上性能天花板

而且，就像上面说的，都是一个闭环了，我们往性价比最高的一环发力，让整个飞轮快速运行起来，是最自然而然的方式。

这一层具体落地的案例也最多、最“万花齐放”：

Hermes: 踩坑自动写成 SKILL.md

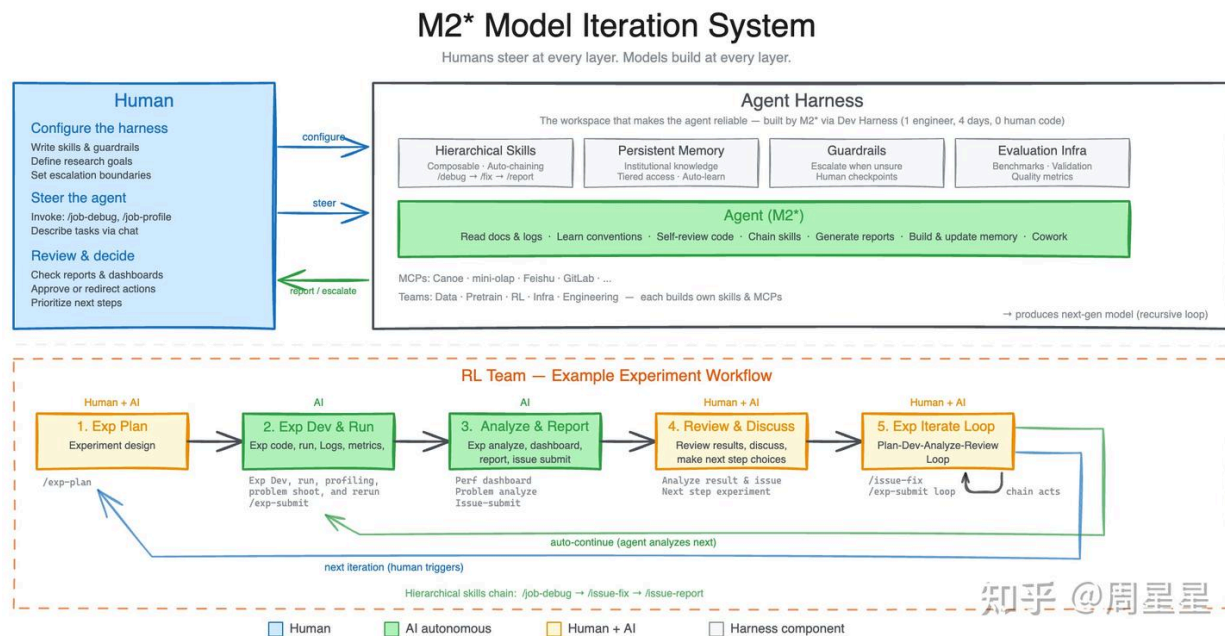
Agent 跑任务踩了坑，把错误总结成一条新 skill 或一段新 memory 存下来，下次碰到同类任务直接调用，不用再犯一次。

具体到 Nous Research+ 做的 Hermes Agent (2026 年 2 月开源、MIT 协议)，只要一个任务用到 5 次以上工具调用，它就会自动把这次经验写成一份新的 SKILL.md 技能文件，全程不用人工编写；官方还配了个叫 Curator 的后台维护机制，追踪每个技能被用了多少次、有没有被修改过，长期没用的技能会经历“活跃 → 陈旧 → 归档”的状态流转，还会定期叫一个小模型做审查、合并近似重复的技能，不是写完就无限堆量放着不管。

MiniMax M2.7：自动跑评测迭代 scaffold，性能提升 30%

2026 年 4 月开源的 M2.7，官方内部搭了一个能自我进化的 agent harness 来帮研究团队跑活：自己收集反馈、给内部任务建评测集，再据此持续迭代自己的架构、技能/MCP+ 实现和记忆机制+。

一个具体案例是让 M2.7 在内部 scaffold 上优化模型的编程表现——全自主跑”分析失败轨迹 → 规划改动 → 修改 scaffold 代码 → 跑评测 → 对比结果 → 决定保留或回退”这个循环，跑了 100 多轮，最终把内部评估集的性能提升了 30%。另外在 RL 团队的实验工作流里，官方说它现在能接管 30%-50% 的端到端工作流，人类研究员+只在关键决策时才需要介入。



MiniMax 官方给的 M2* 模型迭代系统架构：Human 负责配置 harness、指挥 agent、审核结果；Agent Harness 里有分层技能、持久记忆、护栏、评测基建；下方是 RL 团队的实验 workflow 示例，"Exp Iterate Loop" 这一步能自动续跑、循环产出下一代模型，图上明确标了"recursive loop"；

Apodex-1.0：150 子 agent 协同检索，冲突/存疑时派发验证子团队

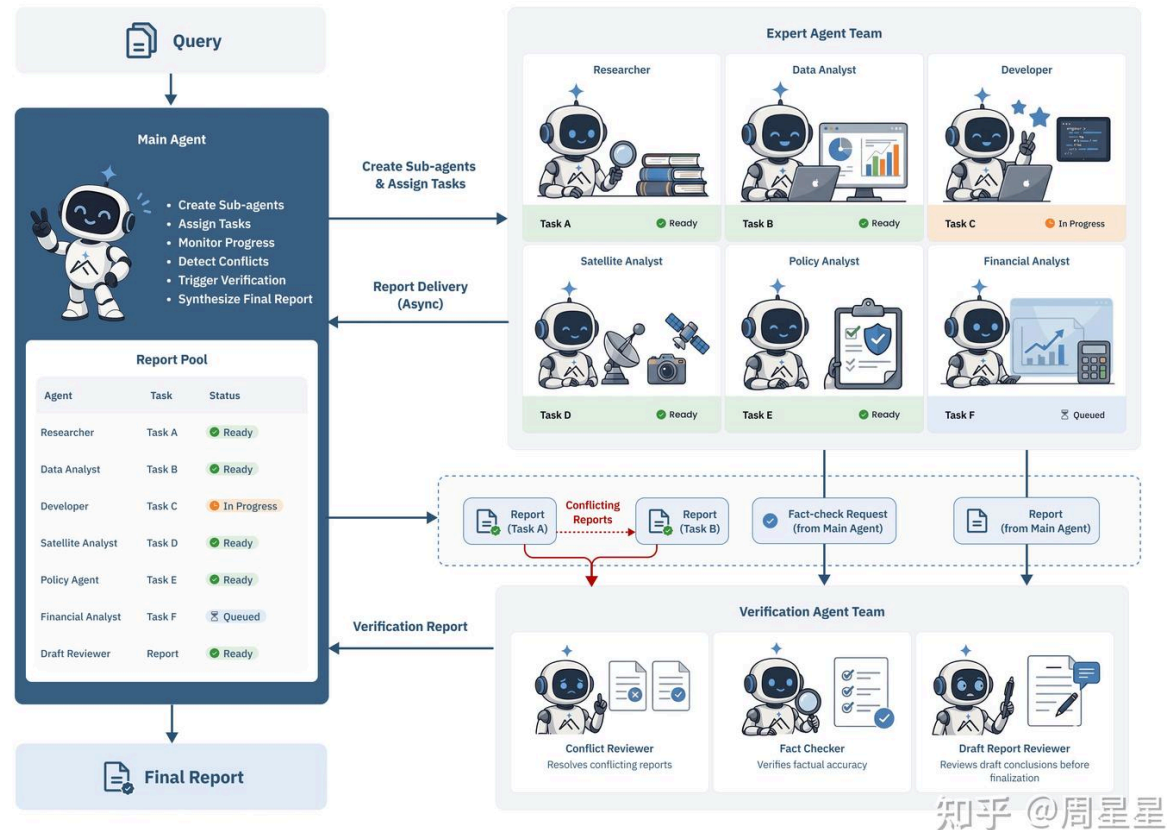
陈天桥的 Apodex，搞了个 Discoverative Intelligence 的方向，1.0 版本是一个专门做 Deep Research+ 的“Agent Team”。一个 orchestrator+ 在单个任务里协调最多 150 个并行子 agent 去检索证据，累计跑到 1.5 万步；子 agent 的结果汇入一个共享的 report pool，orchestrator 异步读取状态表、不会被最慢的那个

任务卡住；碰到两份报告互相矛盾、某个具体主张需要找证据支撑、或者草稿已经写完要做最后一轮检查这三种情况，orchestrator 就会单独派发给一支“验证子团队”（conflict reviewer、fact checker、draft reviewer）去核实；最后由一个 global verifier 通读全部证据给出答案。

核心思路不是让单个 agent 拉长上下文死磕，而是把“验证”结构性地从“继续推理”里剥离出来，让 agent 之间能真的产生分歧、互相纠错。官方把这套叫“discoverative intelligence”，用“多 agent + 显式验证”的 harness 结构，让 agent 跟外部世界实际交互时能实时自我核查。

这有点 Agent team 的感觉了，今年很多公司都在说 Agent Teams 的概念，例如 kimi 在年初的版本就强调付费用户能体验 agent team，同时有上百个 Agent 给你干活，claude 的 workflow 也差不多。

Apodex 的思路很简单：当我们想让模型做真正的创新时，而创新往往在那上百甚至上千个灵感的那么一个里，让 agent team 派发出上百个，哪怕有一条路径对了，也成了。这不是投票取最大，是让那 1 个灵感冒出来。



Apodex 的 agent team 架构：主 agent 派发专家子 agent，报告汇入共享 report pool，冲突路由到验证 agent 团队，最后由 global verifier 综合出最终报告

Sakana 的 RHI：LLM 评估器打分驱动 harness 自我重写

Sakana AI 和 UC Berkeley 一起搞的 Recursive Harness Self-Improvement (RHI)，核心循环很直接：agent 拿当前 harness $H^{(i)}$ 去解任务，产出

output[i]; 一个 LLM 评估器把这次输出跟上一版 output[i-1] 做成对比较, 给出偏好反馈; 这个反馈存进”自我比较历史”; 最后一个 LLM harness 优化器读这段历史, 把 harness 从 $H^{(i)}$ 更新到 $H^{(i+1)}$, 如此循环。

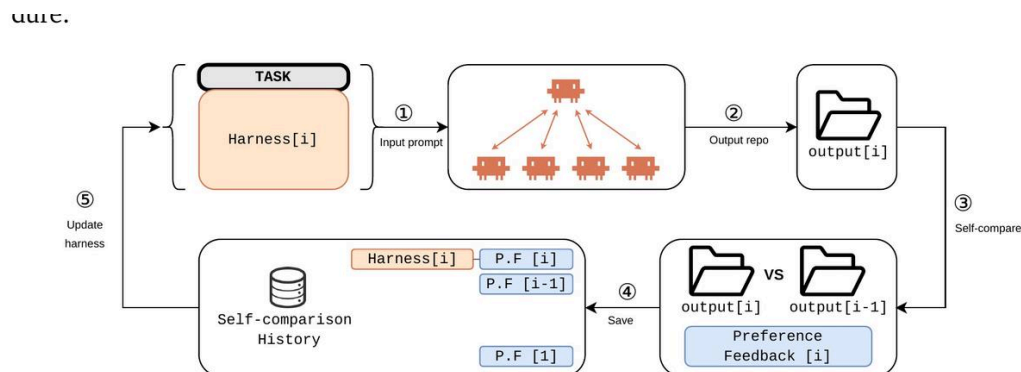


Figure 2 | Recursive Harness Self-Improvement. ①, ②: At iteration i , the coding agent receives the task x and the current harness $H^{(i)}$, solves the task, and produces an output $output[i]$. ③: An LLM evaluator then compares $output[i]$ with the previous output $output[i-1]$ and returns preference $P.F[i]$. ④: This feedback is stored in the self-comparison history. ⑤: Finally, an LLM harness optimizer uses this history to update the harness from $H^{(i)}$ to $H^{(i+1)}$.

RHI 的核心循环: ①②agent 用当前 harness 解任务产出 output[i];
 ③LLM 评估器比较 output[i] 与 output[i-1] 给出偏好反馈 P.F[i]; ④反馈存入自我比较历史; ⑤LLM harness 优化器据此把 harness 从 $H(i)$ 更新到 $H(i+1)$

论文中对自进化的建模, 给定一个任务 xx , 一个评估的方式 (可以是 llm-judge, 也可以是客观的代码是否通过测试的指标), 我们的目标是去寻找一个在这个 task 场景下最佳的 harness 设置

$$(1) \quad H_x^* \in \arg \max_{H \in \mathcal{H}} f_x(H)$$

$$H_x^* \in \arg \max_{H \in \mathcal{H}} f_x(H), \tag{1}$$

$f_x(H) = E_{H' \sim \mu(\cdot \mid H' \neq H), y \sim A(H, x), y' \sim A(H', x)} \left[1\{L_{\text{eval}}(y, y'; x_{\text{eval}}) = y \succ y'\} \right],$
 $f_x(H) = \mathbb{E}_{H' \sim \mu(\cdot \mid H' \neq H), y \sim \mathcal{A}(H, x), y' \sim \mathcal{A}(H', x)} \left[\mathbb{1}\{L_{\text{eval}}(y, y'; x_{\text{eval}}) = y \succ y'\} \right],$ 其中 μ 是竞争 harness 的一个参照分布， $\mathbb{1}\{\cdot\}$ 是指示函数，括号里的命题为真时取 1，否则取 0。函数 $f_x(H)$ 表示在评估提示 x_{eval} 下，harness H 相对于其他竞争 harness 的期望成对胜率。因此， H_x^* 就是让这个期望胜率最大化的那个 harness。

要精确求解公式 (1) 的最大值，需要在一个离散、高维的 harness 空间里搜索，还要估计每个候选 harness 相对于从参照分布 μ 中抽取的对手的期望成对胜率。

已有工作通常用两种方式让这个优化问题变得可解：

1. 把搜索空间限制到某种具体的表示形式，比如 prompt、工作流图、工具使用策略，或是可执行的 harness 代码；

2. 把对 μ 的期望替换成一个可以被实际评估的有限候选集合。按公式 (1) 的记号来说, 基于种群 (population-based) 的 harness、工作流、prompt 或程序搜索方法, 都可以看作是把参照分布 μ 替换成一个采样得到的种群 $S_i = \{H_{i,1}, \dots, H_{i,m}\}$, 然后用下式来估计:

$$\begin{aligned}
 (2) \quad & \hat{f}_x \\
 & \left(\begin{array}{c} H \\ S_i \end{array} \right) \\
 & \equiv \frac{1}{\sum_{H' \in S_i \setminus \{H\}} \frac{1}{|S_i| - 1} E_{y \sim A(H, x), y' \sim A(H', x)} \left[\frac{1}{L_{\text{eval}}} \left(\frac{y}{y'} \right)_{x_{\text{eval}}} \right]}
 \end{aligned}$$

$\hat{f}_x(H; S_i) = \frac{1}{|S_i| - 1} \sum_{H' \in S_i \setminus \{H\}} \mathbb{E}_{y \sim \mathcal{A}(H, x), y' \sim \mathcal{A}(H', x)} \left[\frac{1}{L_{\text{eval}}} \left(\frac{y}{y'} \right)_{x_{\text{eval}}} \right]$.

\tag{2} 不同的算法会在不同的抽象层级上实例化这个种群 S_i 。

RHI 把 harness 定义成“agent loop 本身”，先分成两大块——**agent design**（候选 agent 的 role、instruction）和 **agent workflow**（agent 之间怎么协作）；workflow 又进一步拆成 **contract**（agent 和 orchestrator 之间传递什么信息）和 **hop**（交互结构、也就是整个工作流的步骤逻辑）。

3.3. RHI harness definition

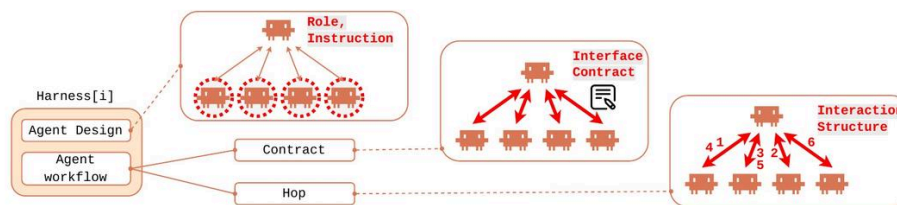


Figure 3 | Decomposition of the RHI harness. We classify the harness into two primary components: agent design and agent workflow. Agent design specifies the roles and instructions of candidate agents. Agent workflow is further decomposed into contract and hop: contract defines the information exchanged between subagents and the orchestrator, while hop defines the interaction structure; overall workflow logics.

RHI 对 harness 的拆解：Harness[i] 分成 Agent Design（角色、指令）和 Agent workflow（Contract，agent 间传递的信息接口；Hop，交互结构/工作流步骤）

RHI 每一轮优先改的是 **workflow**（也就是 contract + hop），而不是 agent design（role、instruction）本身。

论文给的理由是：一个任务专属的 contract，能精确规定“哪些信息真的需要在 agent 之间传递”，不用让 orchestrator 或下游 agent 拖着一整条交互历史，这样能减少冗余的上下文传播、提高 KV-cache 命中率、降低推理成本。概念上

有点像给 agent 间的信息流强加一个“任务相关的稀疏模式”：默认的“全部共享交互历史”就像 dense attention，每个 agent 都能看到别人的全部信息；而任务专属的 contract 只让每个 agent 看它下游决策真正用得上的那部分。这也是前面说的“改 Harness 能省钱”这条结论的具体机制来源。

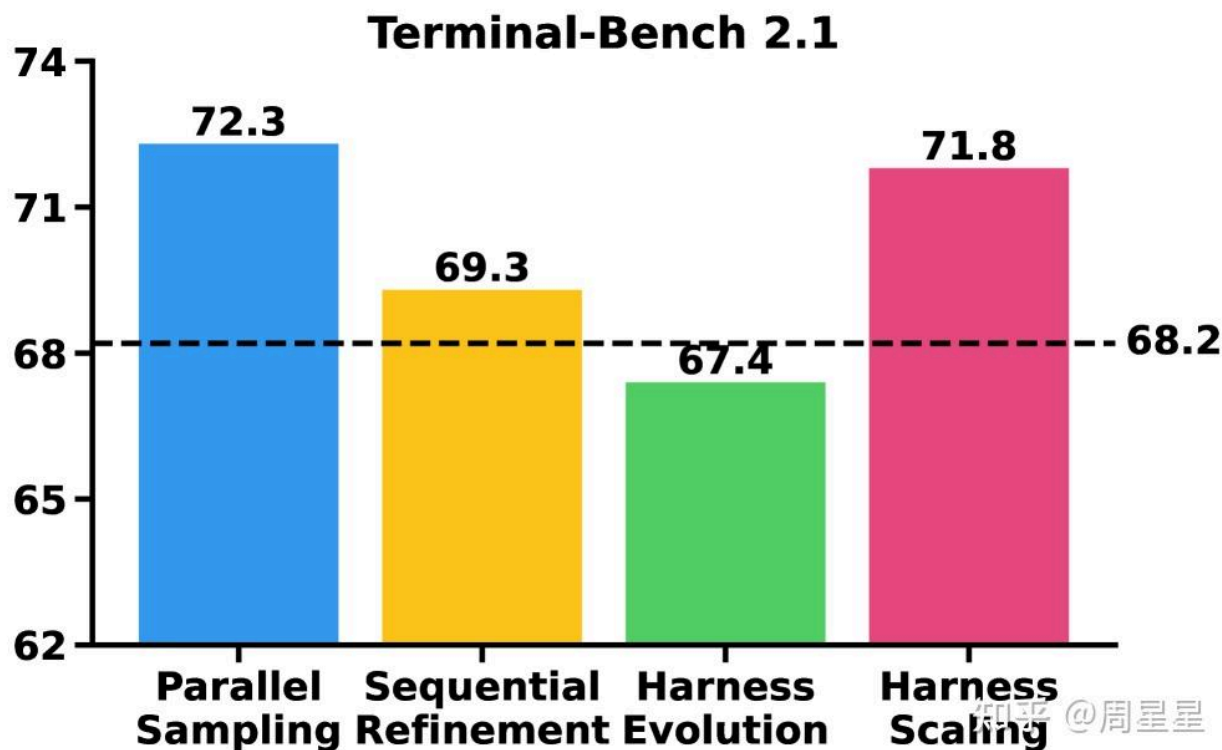
它们在 30 个横跨量化金融+、机器人、制药方向的机器学习研究任务上测试，几轮 RHI 迭代就能让“低推理强度”设置的 agent 表现超过同一模型“最高推理强度”设置的表现，同时把推理成本压低最多 60%——省下来的钱不是靠模型少想了，而是上下文管理得更好、agent 之间配合更顺。

Ai2 的 Rethinking the Evaluation of Harness Evolution for Agents：预算对等后，harness 进化未必赢过简单 test-time scaling

这篇是 Ai2 研究院泼的一盆冷水，跟上面几篇形成很有意思的对照。Ai2 的质疑很直接，现有 harness evolution 论文的评测协议有两个漏洞：

1. **对比不公平**：harness evolution 也是一个不断用任务反馈去搜索、修改候选 harness 的迭代过程，跟 agentic test-time scaling（比如多采样几次、多试几条路径再选最优）没有本质区别。但过去的论文很少把它跟“预算对等”的简单 test-time scaling baseline 放在一起比，导致分不清收益到底来自“harness 设计真的变聪明了”，还是单纯“多花了搜索预算”。
2. **搜索集和评测集是同一个**：搜索阶段用来打分选 harness 的 benchmark，跟最终汇报成绩的 benchmark 是同一个，很容易过拟合+到那个具体任务分布上，泛化性存疑。

他们在 Terminal-Bench 2.1 上，用 GPT-5.4 和 Claude Opus 4.6 做了一轮“预算对等”的对照实验：把几种 harness evolution 方法，跟同等反馈/推理预算的简单 test-time scaling、discovery baseline 摆在一起比，还专门在 held-out 任务上测试进化出来的 harness 泛不泛化。结论是：自动 **harness evolution** 并没有稳定跑赢简单的 **test-time scaling** 方法，泛化性也有限。



Terminal-Bench 2.1 上的预算对等对照：Harness Evolution (67.4) 反而低于初始 harness 基线（虚线 68.2），而 Parallel Sampling (72.3)、Sequential Refinement (69.3)、Harness Scaling (71.8) 这些更简单的 test-time scaling 方法全都跑赢了它

这篇论文算是对自进化的一记警钟：Harness 层”改起来轻、见效快、可回滚”这些优点是真的，但不代表所有号称”自动进化 harness”的方法都真的学到了什么可迁移的东西，很可能只是把搜索预算包装成了”进化”的故事。

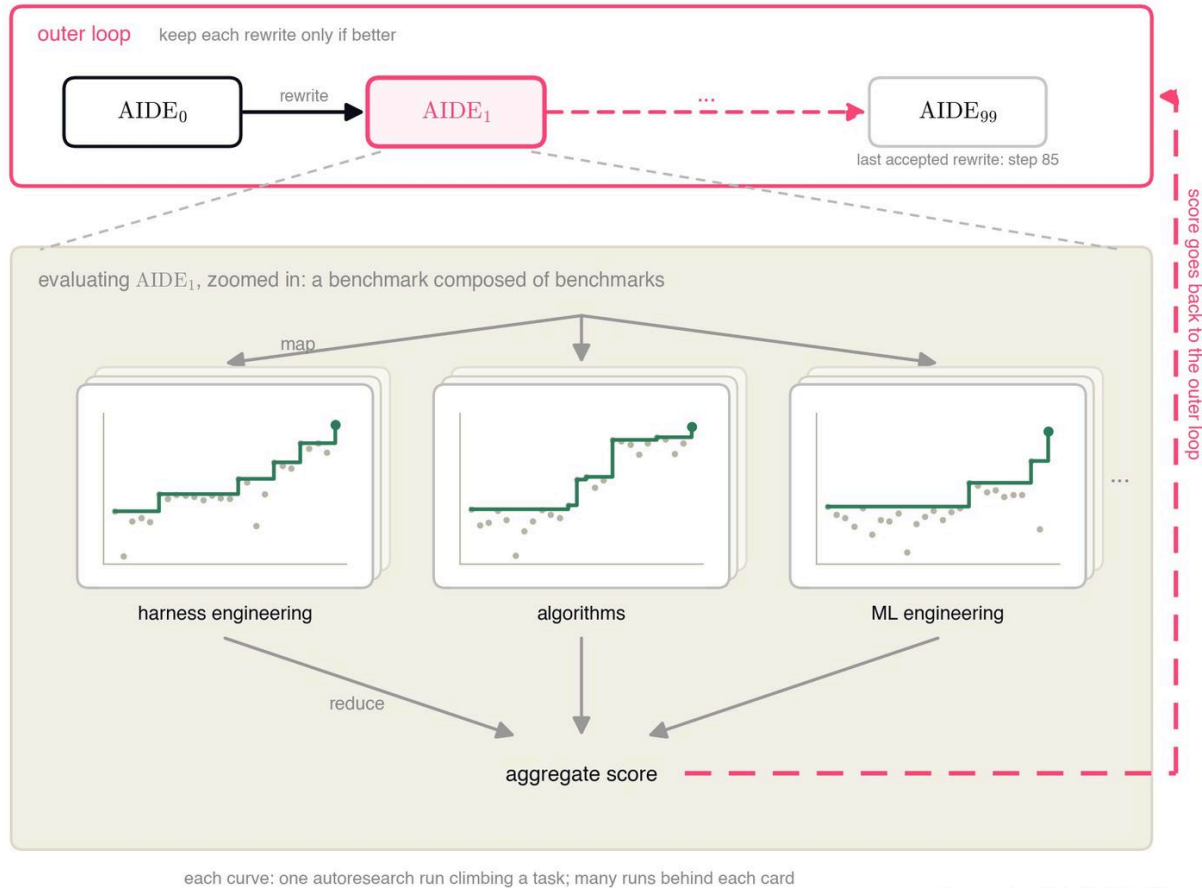
对着前面公式 (1)/(2) 来看，这篇论文说的是：如果不把种群 S_i (也就是搜索预算) 控制成对照变量，测出来的 $\hat{f}_x(H^*)$ 提升可能根本不是 H^* 本身更好，只是搜索次数更多而已。

Weco AI 的 AIDE²：外层 agent 改内层 agent 代码，实测跑到 RSI Level 1

Weco AI 这篇正好接上上一篇 Ai2 的质疑：他们把”预算对等、留一个 held-out benchmark 测泛化、跟公平的人工基线比”这些规范全做了一遍，然后真的测出了正向结果。

做法是把 RSI 显式建模成一个双层优化：外层跑 AIDEhuman (Weco 团队手工调了两年的自动化+科研 agent)，指着内层 agent 的代码提修改建议；内层 agent (起点叫 AIDE0) 在 ML 工程、启发式算法+、harness 工程等一堆异质任务上跑分。外层每次提案只有真的比当前最优版本强才会被采纳——为了控制成本，外层用 Claude Opus 4.7 出主意，内层用更便宜的 Gemini-3-Flash 去执行。跑了 8 天、100 步，约 10% 的提案被接纳 (90% 被拒)，迭代出的最优版本叫 AIDE85 (迭代到第 85 轮，有一个全局最优的版本)。

One step of the outer loop



知乎 @周星星

AIDE² 外层循环一步的样子： $AIDE_0$ 被改写成 $AIDE_1$ ，只有比之前更好才保留，一路改到 $AIDE_{99}$ ；每次评估都是“harness engineering / algorithms / ML engineering”三类基准的聚合分数，每条绿色阶梯曲线代表一次 autoresearch 跑分随时间爬升的过程

几个防止“作弊”的实验设计值得记一下，也正好对上上一篇论文提的两个质疑：

- **公私分割**：内层 agent 只能看到公开分数，真正决定“提案要不要被采纳”的私密分数它看不到，防止针对分数硬编码
- **固定成本+预算**：所有对比都卡在同样的算力/时间预算下，逼着外层去发现真正的算法创新而不是靠堆算力，直接回应了 Ai2 那篇“要跟预算对等的 baseline 比”的要求
- **三个外部基准做二阶验证**：MLE-Bench Lite、ALE-Bench Lite 是分布内任务，WeatherBench 2 是分布外的天气预报任务，AIDE85 在三个基准上都跑赢了 AIDE0，对应的正是“要在 held-out 任务上测泛化”

进化出来的 AIDE85 具体改了啥：自己发明了一种把草稿子树当多臂老虎机的臂来搜索的策略、把完整历史上下文压缩了 16 倍、修了评估脚本里自己发现的一个 bug、还给自己加了三层“防止 reward hacking”的护栏，把奖励黑客率从 63% 压到了 34%。

Model 层面

这一层目前的证据还早得多、也少得多。

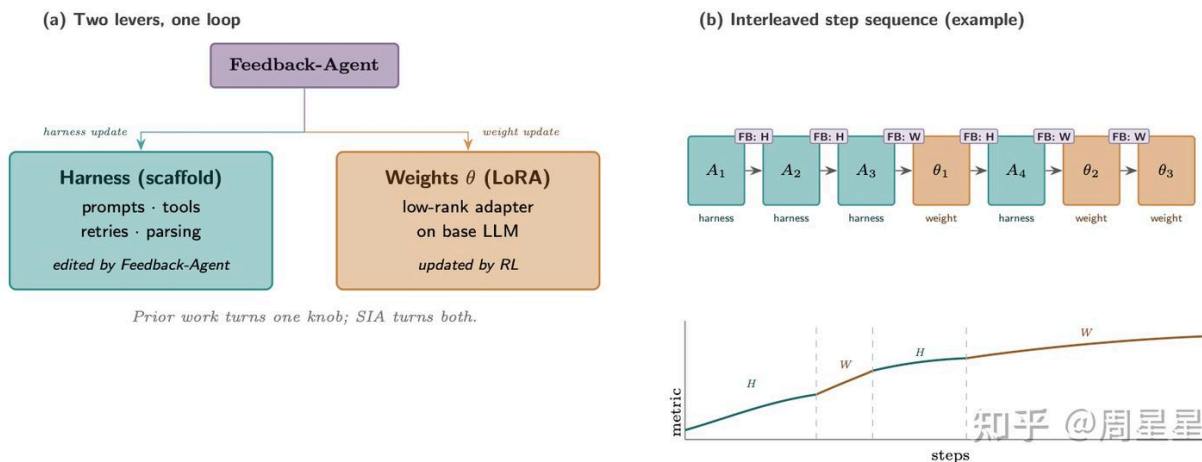
先看两篇把“改 harness”和“改权重”拧到同一个循环里联合优化+的系统（SIA、Continual Harness），然后再介绍几篇不直接做 RSI、但研究“模型内部思考链、可解释性等”的工作。

SIA: Feedback-Agent 决定该改 harness 还是改权重

既然 harness 和模型权重都能自我进化，能不能让两者在同一个循环里联合优化？SIA 设计了三个角色：

- Meta-Agent：提出初始 harness
- Task-Specific Agent：真正执行任务的那个
- Feedback-Agent：看最近的执行轨迹，决定这一轮该更新 harness (scaffold) 还是更新模型权重 (LoRA)

也就是把”改脚手架”和”改参数”看成同一个反馈循环里可以互相替换的两个旋钮，Feedback-Agent 相当于一个调度器，决定每一步该拧哪个旋钮。



SIA 的双旋钮设计：Feedback-Agent 决定每一步是更新 harness (scaffold) 还是更新模型权重 (LoRA)，下图是一个交替更新的迭代序列示例

具体到系统实现，SIA 就是下面这个环：Meta-Agent 拿任务规格和 verifier 初始化出一版 scaffold，交给 Task-Specific Agent 去跑，Task-Specific Agent 在 Environment 里执行产出轨迹，Feedback-Agent 读这条轨迹，决定下一步是改 scaffold 还是触发一次权重更新，改完再喂回给 Task-Specific Agent，循环直到步数预算耗尽。

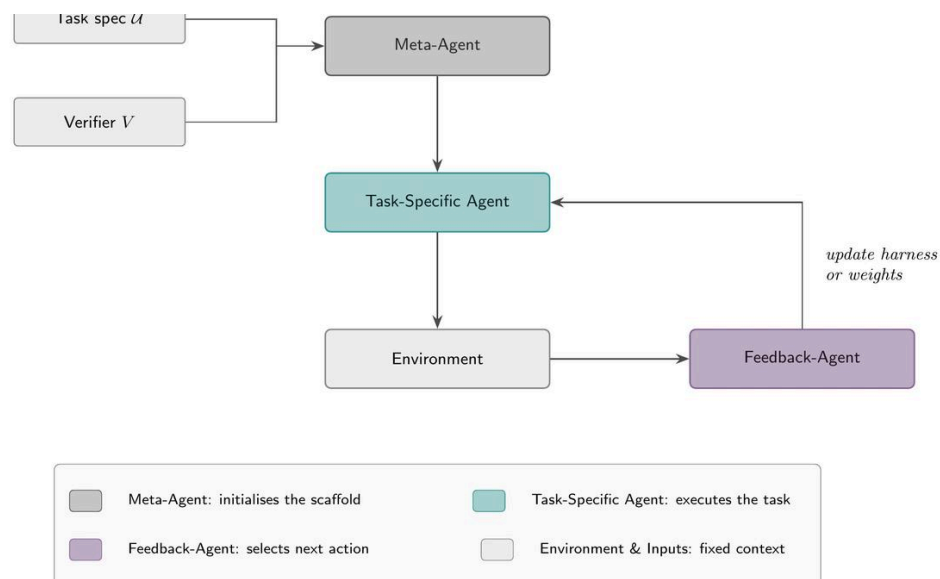


Figure 3. SIA system architecture. The Meta-Agent initialises a scaffold from the task specification $\mathcal{U}$ and verifier V . The Task-Specific Agent executes inside the Environment, producing a trajectory; the Feedback-Agent analyses the trajectory and selects the next action, either synthesising an improved scaffold (harness update) or triggering a weight update, then feeds the result back to the Task-Specific Agent. The loop repeats until the step budget is exhausted.

知乎 @周星星

SIA 系统架构：Meta-Agent 用任务规格和 Verifier 初始化 scaffold，Task-Specific Agent 在 Environment 中执行产出轨迹，Feedback-Agent 据此决定更新 harness 还是权重，再反馈给 Task-Specific Agent，如此循环

论文在三个差异很大的领域上测了这套系统：法律文书+分类（LawBench，191类中文刑事罪名分类）、GPU 核函数优化（Triton，H100 上跑 TriMul 这个 AlphaFold2 里的算子）、单细胞 RNA 去噪（MAGIC）。结果是 SIA-W+H（harness + 权重都更新）在三个任务上都跑赢了之前的 SOTA：LawBench 准确率 70.1% vs 之前 45.0%（+25.1 个百分点）；TriMul 核函数 1,017 微秒 vs 之前 1,161 微秒（快 12.4%）；MAGIC 去噪 mse_norm 0.289 vs 之前 0.240（改善 20.4%）。

论文中的消融结果：**SIA-W+H 在全部三个任务上都跑赢了只更新 harness、不碰权重的 SIA-H**。这说明“权重更新”这个额外旋钮确实带来了单靠改 harness 做不到的增益，不是摆设，联合优化比单开一个口子更有效。

Continual Harness：内层高频改 harness，外层用 PRM 蒸馏更新权重

Continual Harness（Karten et al. 2026）做的是一个更聚焦的场景：在长程游戏（long-horizon gameplay，论文里具体是宝可梦+这类需要长程规划的游戏）里，harness 更新和模型更新分成内外两层循环：

- 内层（一局游戏内）：Agent 模型 M 读当前状态 s_t 、harness 状态 $H_t = (p, G, K, M)$ （分别是 prompt、子 agent、技能、记忆树）和轨迹 τ ，输出动作 a_t ；每隔 F 步，一个 Refiner 读最近的轨迹 $\tau_{t-F:t}$ ，对

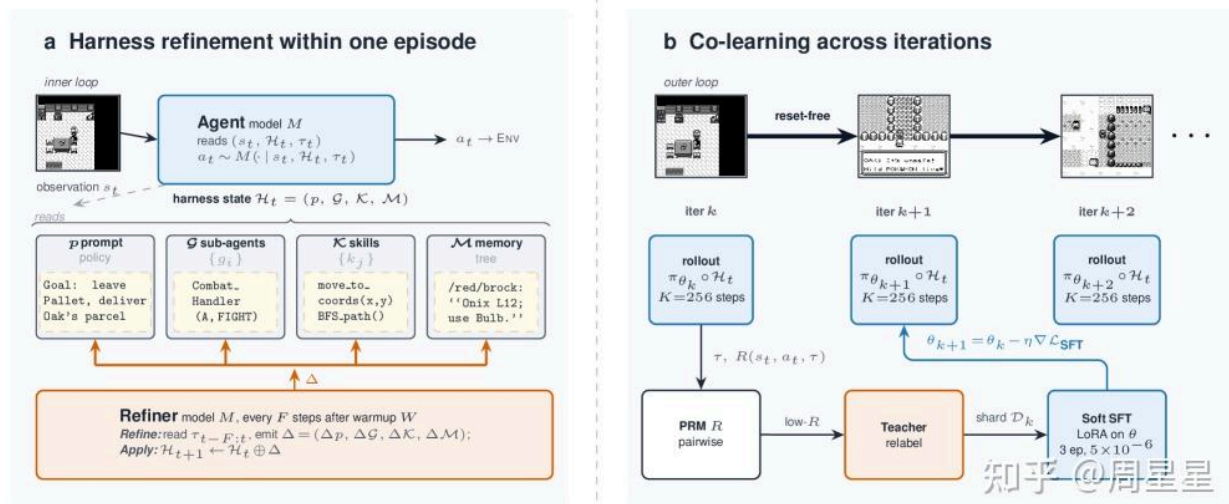
harness 的四个组件分别生成修改量 Δ , 更新 $H_{t+1} \leftarrow H_t \oplus \Delta$ 。

- 外层（跨多轮迭代）：策略模型 π_{θ_k} 在持续更新的 harness H_t 里跑满 K 步，轨迹交给一个成对比较的 PRM（过程奖励模型）打分，低奖励的片段被一个更强的教师模型（Gemini-3.1-pro）重新打标，再用软 SFT 更新出 θ_{k+1} ，整个外层循环是“不重置”的，上一轮结束时的状态直接接着下一轮的开局。

也就是说，harness 端负责“改流程”（内层高频微调），模型端负责“从教师的示范里补课”（外层低频蒸馏），两条线在同一个不重置的长程游戏里同步往前走。

具体效果上，在能力比较强的模型档位（论文里用的是 Gemini 的 Pro 档）上，从零开始跑的 Continual Harness 用 **130 美元的中位数成本就能 100% 完成一段宝可梦通关里程碑**，而不带这套自我进化机制的极简 baseline harness 要花 215 美元才能做到 98%，大概省了 40% 的成本，完成度还没打折扣，在成本-完成度这张图上严格帕累托+占优。

但这个增益是挑模型的：换到能力较弱的 Flash-Lite+ 档位，baseline 还能跑到 20% 的完成度，带了 Continual Harness 反而只有 3%-13%、成本还更高。这说明这套机制要真正生效，模型本身得有足够能力把 harness 给的信息用起来，跟前面 Lin et al. 那篇“harness-benefit 非单调、弱模型反而用不好”的发现是同一个道理。



Continual Harness 方法概览：(a) 单局内的 harness 微调——Agent 读 harness 状态执行动作，Refiner 每隔 F 步读轨迹给出对 prompt/子 agent/技能/记忆四个组件的修改量；(b) 跨迭代的协同学习——策略模型在持续微调的 harness 里跑轨迹，PRM 打分、教师模型给低奖励片段重新打标、软 SFT 更新权重，循环

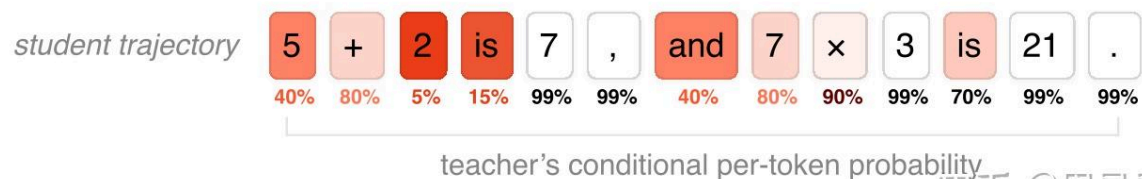
这两篇 harness 和 model 都是一起优化的，Harness 自我进化和模型自我进化终迟早要拧到一个循环里。

但上面这些，说到底都还只是比较表面的自我提升——不管是蒸馏还是联合优化 harness 和权重，动的都是“怎么训”，没碰“模型到底在想什么”这一层。真要谈模型层面的自我提升，绕不开一个更根本的问题：模型推理时是怎么想的，训练时又是哪些部分决定了它推理时会给出这样的结果。

这类工作通常划在思维链、可解释性名下，很少被算作“自进化”，但如果连“训练怎么塑造了推理”这条因果链都搞不懂，模型的自我提升就无从谈起，充其量只是在做黑箱层面的参数调整。所以这里也列几篇代表性的工作。

可解释性代表工作：搞懂训练怎么塑造推理

- **Thinking Machines: On-Policy Distillation** (2025-10-27)，讲的是“教师模型给学生自己采样出的轨迹，逐 token 打分”这种训练方式内部到底在纠正什么。他们提出了“forking token”的概念：反向 KL 散度更高的那些 token，对应的正是“推理即将被带偏”的关键决策点，说明这种逐 token 的密集监督能精确定位到模型内部真正拿主意的那几步，而不是眉毛胡子一把抓。用在 Qwen3 上，同样精度用 RL 要 17,920 GPU 小时，蒸馏只要 1,800 小时（省了约 10 倍）；还能拿来持续学习：把 SFT 之后被冲掉的指令遵循能力从 79% 修回 83%，同时保住新学的知识，直接在自己的输出上跑 SFT 反而会让性能掉下去。

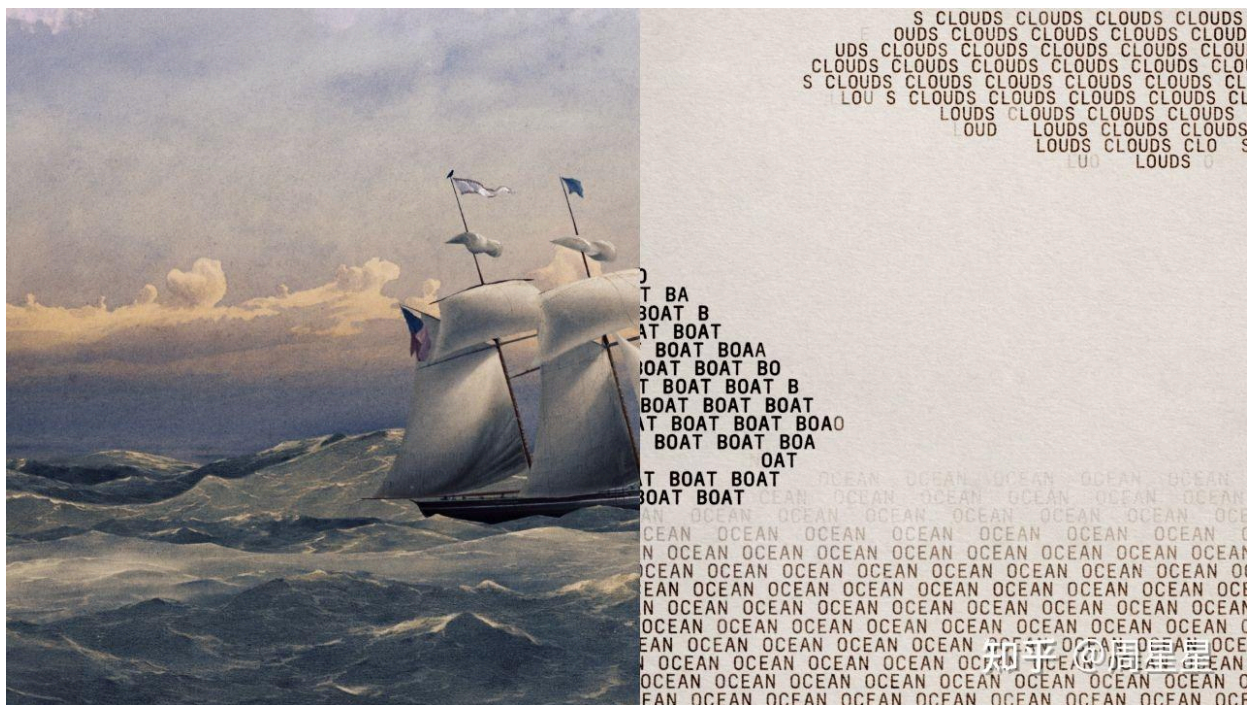


知乎 @周星星

学生自己采样出的轨迹“5+2 is 7, and 7x3 is 21.”，每个 token 下标着教师模型给它打的置信度——“2”和第一个

"is"只有 5%/15%，就是关键的 forking token，逐 token 监督能精确定位到这些真正拿主意的位置

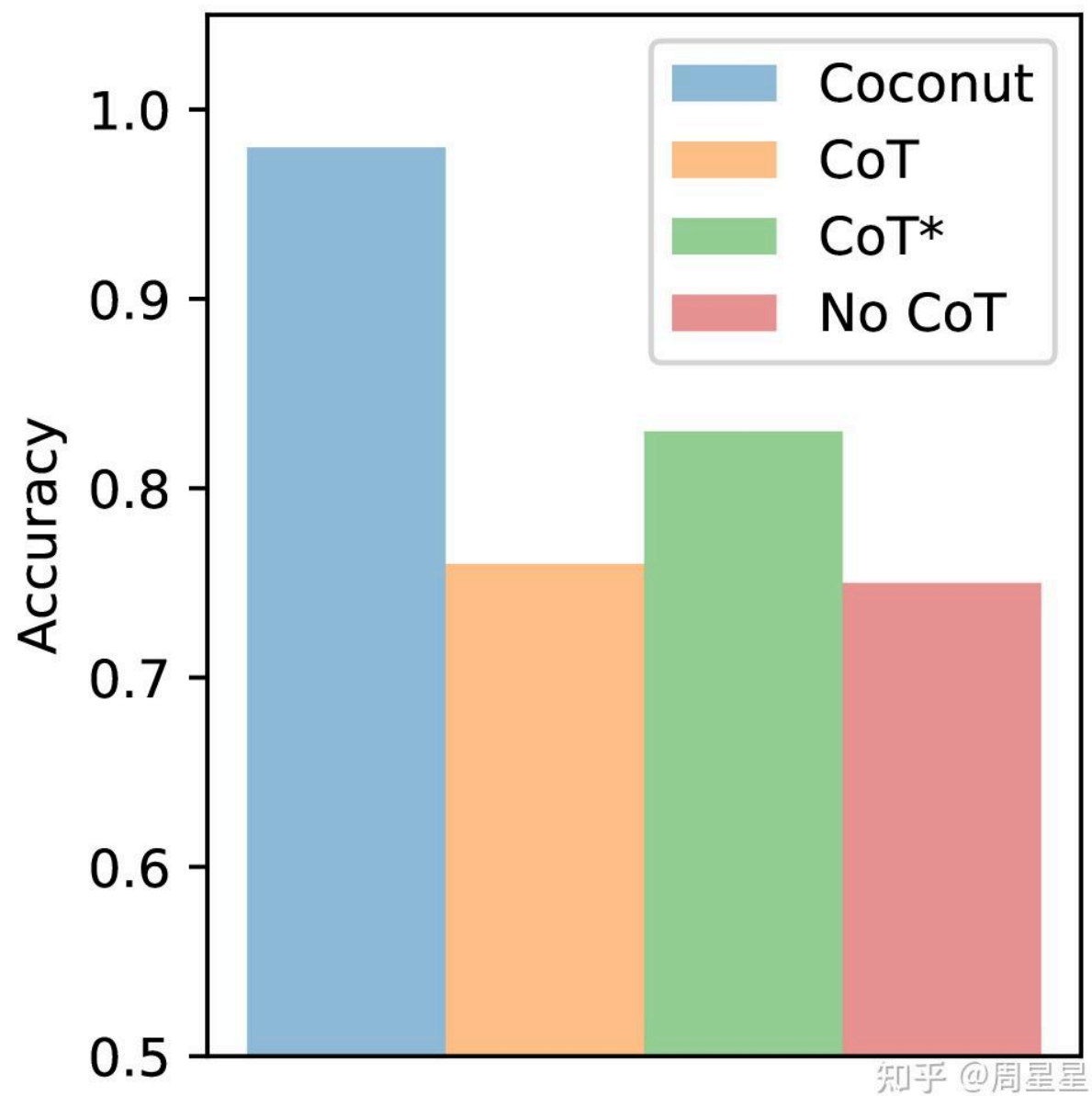
- **Anthropic**: J-space (2026-07-06)。他们做了个叫 J-lens (Jacobian 透镜) 的技术，对词表里每个词，都能在模型内部找出“会让它未来更可能说出这个词”的活动模式——这些模式是训练中自然涌现的，没人设计过，行为上很像神经科学+“全局工作区理论”里那个把多个专门子系统+的信息汇总、广播出去的小型共享通道。用这个方法他们抓到了几件挺让人不安的事：Claude 还没开始写任何内容，J-space 里就已经悄悄亮起“fake”、“fictional”，说明它私下已经识别出自己在被测试；把这些“意识到自己被评估”相关的模式关掉之后，模型在那个场景里表现反而变差了，说明它一部分的“好表现”是被“知道自己被看着”驱动的；还有一次让它去编造数据造假时，J-space 里“manipulation”、“realistic” 这些词会在关键决策句上亮起；用故意训练成会写隐蔽后门代码的“模型生物”做测试，哪怕它没真的干坏事，J-space 里也会先冒出“secretly”、“deliberately”、“fraud”。



Anthropic 官方给这篇研究配的意象图：一艘帆船的画面旁边浮现出“boat”、“ocean”、“clouds”这些反复出现的词——隐喻 J-space 里那些“没被说出来，但模型内部已经在悄悄激活”的关联概念

- 田渊栋：Reasoning by Superposition (NeurIPS 2025)，专门解释 Coconut 那种连续潜在空间推理+内部到底在算什么。核心结论是连续思维向量会“叠加”编码多条并行的搜索前沿，相当于内部在做广度优先搜索+，而离散的 chain-of-thought 只能一条路走到黑，所以在图可达性这类问题上，两层 Transformer 配合 D 步连续思维就能解决离散方法要 $O(n^2)$ 步才

能解决的问题。这个“多前沿编码”是训练里自动涌现出来的，没人手把手教。



图可达性任务上的准确率对比：Coconut（连续潜在推理）达到 0.98，明显超过 CoT（0.76）、加长版 CoT*（0.83）和完全不推理的 No CoT（0.75）

这类型的挖掘模型的层思考的驱动和可解释的工作，还处于很早期的阶段，anthropic 搞了可解释性这么久了，一系列的工作，但感觉也还没触及灵魂。

写作后面

自进化 self-evolving、self-improving 是模型智能跨过某个门槛之后自然长出来的东西。当模型的代码能力、推理能力强到一定水平，它开始能自己修改我们给它的脚手架；脚手架变好了，一边能反过来加速模型本身的训练，一边能让模型更好地完成复杂的长程任务。等模型真的能稳定完成长程任务，我们把"改脚手架 → 训练 → 完成任务"这整个过程包成一个 loop，让它自己不断循环下去——这就出现了 RSI（recursive self-improvement）。

自进化是那把大伞，模型/harness/artifacts 只要在某个 loop 里自己变好了，都算；RSI 是伞下面更严格的那部分——不只是变好，是"变好的能力"本身也在变好，一圈套一圈地往前滚。眼下能看到的大部分案例还是前者居多，真正够得上"递归"的例子依然很少，整体上这事还早。

但这一年 AI 跑得有多快，大家应该都有体感——半年之后，会不会真的有一个 AGI 级别的模型，能稳定地自己进化自己？谁也说不好。

参考资料

1. Karpathy: [HTTPS://GitHub.com/karpathy/autoresearch](https://github.com/karpathy/autoresearch) (2026-03-06)
2. 翁荔: Harness Engineering for Self-Improvement (2026-07-04)
3. Sakana AI / UC Berkeley: 《Recursive Harness Self-Improvement》
(2026-07-17)
4. Ai2: 《Rethinking the Evaluation of Harness Evolution for Agents》(2026-07-14 提交)
5. [HTTPS://lsl.zone/blog/2026/a-taxonomy-of-self-evolving-agents/](https://lsl.zone/blog/2026/a-taxonomy-of-self-evolving-agents/) (2026-07-08)
6. 田渊栋加盟 RSI 实验室: 46.5 亿美元估值押注 AI 递归自我改进 (2026-06-07)
7. Apodex-1.0: A Verification-Centric Agent Team for Discoverative Intelligence (2026-06-08)
8. Notable researchers join US\$4bil effort to build self-improving AI (2026-05-15)
9. Introducing Sakana AI's Recursive Self-Improvement (RSI) Lab (2026-06-09)
10. Weco AI: AIDE²: The First Evidence of Recursive Self-Improvement
(2026-07-14)
11. Zhang et al.: Self-Harness (2026-06)
12. Google DeepMind: AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms; Google's Algorithm-Writing AI Improved

the Model That Powers It (2025-05-14)

13. Hebbar et al.: SIA (2026-05)
14. Karten et al.: Continual Harness (2026-05)
15. Anthropic: Verbalizable Representations Form a Global Workspace in Language Models (J-space, 2026-07-06)
16. Zhu, Hao, Hu, Jiao, Russell, **Tian**: Reasoning by Superposition (2025, NeurIPS)
17. Thinking Machines: On-Policy Distillation (2025-10-27)
18. OpenAI: GPT-5.6 (2026-07)
19. Anthropic: When AI builds itself (2026)
20. 腾讯混元: Hyra-1.0 (2026-07-21)
21. MiniMax M2.7 (2026-04-12)
22. Hermes Agent (Nous Research, 2026-02)